



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5

по курсу «Операционные системы»

на тему:

«Буферизованный и небуферизованный ввод-вывод»

Студент ИУ7-62Б
(Группа)

(Подпись, дата)

А. Н. Диваев
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Н. Ю. Рязанова
(И. О. Фамилия)

2026 г.

1 Структуры

Листинг 1 – Структура `_IO_FILE`

```
1 struct _IO_FILE
2 {
3     int _flags;    /* High-order word is _IO_MAGIC; rest is flags. */
4
5     /* The following pointers correspond to the C++ streambuf
6        protocol. */
7     char *_IO_read_ptr;    /* Current read pointer */
8     char *_IO_read_end;    /* End of get area. */
9     char *_IO_read_base;    /* Start of putback+get area. */
10    char *_IO_write_ptr;    /* Current put pointer. */
11    char *_IO_write_end;    /* End of put area. */
12    char *_IO_buf_base;    /* Start of reserve area. */
13    char *_IO_buf_end;    /* End of reserve area. */
14
15    /* The following fields are used to support backing up and undo. */
16    char *_IO_save_base; /* Pointer to start of non-current get area.
17        */
18    char *_IO_backup_base; /* Pointer to first valid character of
19        backup area */
20    char *_IO_save_end; /* Pointer to end of non-current get area. */
21
22    struct _IO_marker *_markers;
23
24    struct _IO_FILE *_chain;
25
26    int _fileno;
27    int _flags2:24;
28    /* Fallback buffer to use when malloc fails to allocate one. */
29    char _short_backupbuf[1];
30    __off_t _old_offset; /* This used to be _offset but it's too
31        small. */
32
33    /* 1+column number of pbase(); 0 is unknown. */
34    unsigned short _cur_column;
35    signed char _vtable_offset;
36    char _shortbuf[1];
37
38    _IO_lock_t *_lock;
39    __off64_t _offset;
40    /* Wide character stream stuff. */
41    struct _IO_codecvt *_codecvt;
```

Листинг 1 (продолжение)

```
39 struct _IO_wide_data *_wide_data;
40 struct _IO_FILE *_freeres_list;
41 void *_freeres_buf;
42 struct _IO_FILE **_prevchain;
43 int _mode;
44 int _unused3;
45 __uint64_t _total_written;
46 /* Make sure we don't get into trouble again. */
47 char _unused2[12 * sizeof (int) - 5 * sizeof (void *)];
48 };
```

Листинг 2 – Структура stat

```
1 struct stat {
2     unsigned long    st_dev;        /* Device. */
3     unsigned long    st_ino;        /* File serial number. */
4     unsigned int     st_mode;       /* File mode. */
5     unsigned int     st_nlink;      /* Link count. */
6     unsigned int     st_uid;        /* User ID of the file's owner. */
7     unsigned int     st_gid;        /* Group ID of the file's group. */
8     unsigned long    st_rdev;       /* Device number, if device. */
9     unsigned long    __pad1;
10    long             st_size;        /* Size of file, in bytes. */
11    int              st_blksize;     /* Optimal block size for I/O. */
12    int              __pad2;
13    long             st_blocks;      /* Number 512-byte blocks allocated. */
14    long             st_atime;       /* Time of last access. */
15    unsigned long    st_atime_nsec;
16    long             st_mtime;       /* Time of last modification. */
17    unsigned long    st_mtime_nsec;
18    long             st_ctime;       /* Time of last status change. */
19    unsigned long    st_ctime_nsec;
20    unsigned int     __unused4;
21    unsigned int     __unused5;
22 };
```

2 Первая программа

2.1 Исходный код

Исходный код первой программы представлен в листинге 3.

Листинг 3 – Исходный код первой программы

```
1 #include <fcntl.h>
2 #include <stdio.h>
3
4 int main()
5 {
6     int fd = open("alphabet.txt", O_RDONLY);
7
8     FILE *fs1 = fdopen(fd, "r");
9     char buff1[20];
10    setvbuf(fs1, buff1, _IOFBF, 20);
11
12    FILE *fs2 = fdopen(fd, "r");
13    char buff2[20];
14    setvbuf(fs2, buff2, _IOFBF, 20);
15
16    int flag1 = 1, flag2 = 2;
17    while (flag1 == 1 || flag2 == 1)
18    {
19        char c;
20        flag1 = fscanf(fs1, "%c", &c);
21        if (flag1 == 1)
22        {
23            fprintf(stdout, "%c", c);
24        }
25        flag2 = fscanf(fs2, "%c", &c);
26        if (flag2 == 1)
27        {
28            fprintf(stdout, "%c", c);
29        }
30    }
31    return 0;
32 }
```

2.2 Результат работы

Результат работы первой программы представлен на рисунке 1.



```
os/lab_05/src master !? >
./1.out
aubvcwdxeyfzghijklmnopqrst
```

Рисунок 1 – Результат работы первой программы

2.3 Анализ полученного результата

В программе выполняется системный вызов `open()`, который открывает целевой файл в режиме «только для чтения» (`O_RDONLY`). В результате этого в таблице файловых дескрипторов процесса резервируется наименьший свободный индекс (как правило, 3, поскольку дескрипторы 0, 1 и 2 заняты стандартными потоками ввода-вывода), а в системной таблице открытых файлов ядра создается единственная структура `struct file`.

Далее библиотечная функция `fdopen()` вызывается дважды, связывая полученный системный дескриптор со структурами `FILE` (указатели `fs1` и `fs2`). С помощью функции `setvbuf()` для обоих файловых потоков в адресном пространстве пользователя выделяются буферы `buff1` и `buff2` размером 20 байт и устанавливается режим полной буферизации (`_IOFBF`).

При первом вызове библиотечной функции `fscanf()` для указателя `fs1` происходит чтение файла: локальный буфер `buff1` заполняется первыми 20 символами исходного файла (от «а» до «t»). При этом указатель текущей позиции `f_pos` в структуре `struct file` открытого файла увеличивается на 20. Из буфера извлекается первый символ «а», а внутренний указатель `_IO_read_ptr` сдвигается на 1 байт.

Затем выполняется первый вызов `fscanf()` для указателя `fs2`. Поскольку оба файловых потока `FILE` опираются на один и тот же файловый дескриптор и, следовательно, на общую структуру `struct file`, ядро начинает чтение с текущей позиции `f_pos = 20`. В буфер `buff2` считываются оставшиеся в файле 6 символов (от «u» до «z»), после чего первый символ из считанных («u») передается в переменную и выводится на экран.

В последующих итерациях цикла программа поочередно извлекает символы из пользовательских буферов `buff1` и `buff2`. Системные вызовы

при этом не производятся, так как данные уже находятся в оперативной памяти процесса. Как только буфер buff2 пустеет (указатели `_IO_read_ptr` и `_IO_read_end` сравниваются), программа продолжает последовательно выводить оставшиеся символы из буфера buff1, пока тот также не будет исчерпан.

2.4 Схема связи структур

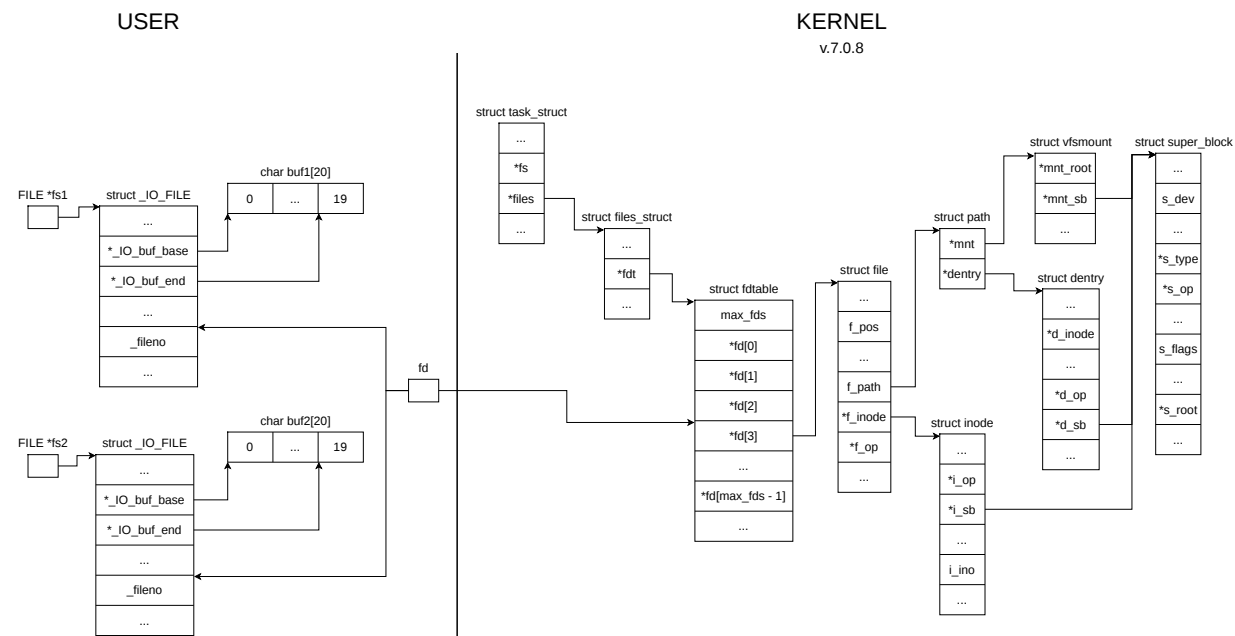


Рисунок 2 – Схема связи структур в первой программе

3 Вторая программа

3.1 Один поток

3.1.1 Исходный код

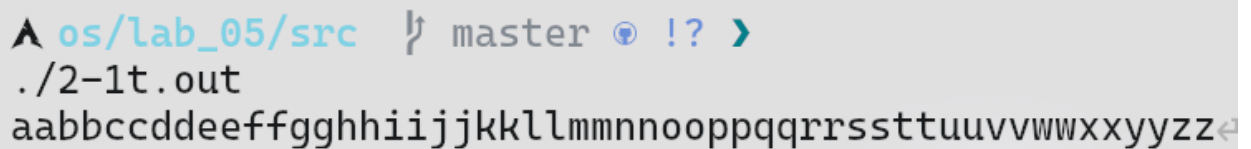
Исходный код второй программы без дополнительных потоков представлен в листинге 4.

Листинг 4 – Исходный код второй программы без дополнительных потоков

```
1 #include <fcntl.h>
2 #include <unistd.h>
3
4 int main()
5 {
6     char c;
7     int fl1 = 1, fl2 = 1;
8
9     int fd1 = open("alphabet.txt", O_RDONLY);
10    int fd2 = open("alphabet.txt", O_RDONLY);
11
12    while ((fl1 == 1) && (fl2 == 1))
13    {
14        if ((fl1 = read(fd1, &c, 1)) == 1)
15            write(1, &c, 1);
16        if (fl1 == 1 && ((fl2 = read(fd2, &c, 1)) == 1))
17            write(1, &c, 1);
18    }
19    return 0;
20 }
```

3.1.2 Результат работы

Результат работы второй программы без дополнительных потоков представлен на рисунке 3.



```
os/lab_05/src master !? >
./2-1t.out
aabbccddeeffgghhiijjkkllmmnnnooppqrrssttuuvvwwxxyyzz
```

Рисунок 3 – Результат работы второй программы без дополнительных потоков

3.1.3 Анализ полученного результата

В данном варианте программы процесс выполняет два независимых системных вызова `open()` для одного и того же файла в режиме «только для чтения» (`O_RDONLY`). В результате в таблице открытых файлов процесса резервируются два уникальных файловых дескриптора (`fd1` и `fd2`). На уровне ядра операционной системы в системной таблице открытых файлов создаются две независимые структуры `struct file`, каждая из которых ссылается на один и тот же `inode` целевого файла.

Ключевой особенностью данной реализации является то, что каждая из созданных структур `struct file` обладает собственным, независимым от другой указателем текущей позиции чтения `f_pos`, который при успешном открытии файла инициализируется нулем.

В рамках цикла `while` процесс осуществляет чтение данных, поочередно передавая системному вызову `read()` сначала дескриптор `fd1`, а затем `fd2`.

При обращении через `fd1` ядро считывает один символ с позиции `f_pos` первой структуры `struct file` и инкрементирует её значение на 1.

При последующем обращении через `fd2` ядро считывает символ, опираясь на `f_pos` второй структуры `struct file`, которая также изначально равна нулю, и аналогично увеличивает её на 1.

Поскольку файловые смещения независимы, через оба дескриптора процесс читает содержимое файла с самого начала. Вывод прочитанных символов в стандартный поток выполняется строго последовательно в рамках одного главного потока исполнения. Благодаря этому состояние гонки отсутствует, а результат программы полностью детерминирован: каждый символ английского алфавита из файла выводится на экран ровно дважды подряд (формируя последовательность вида «aabbcc...»), пока оба указателя `f_pos` не достигнут конца файла.

3.2 Один дополнительный поток

3.2.1 Исходный код

Исходный код второй программы с одним дополнительным потоком представлен в листинге 5.

Листинг 5 – Исходный код второй программы с одним дополнительным потоком

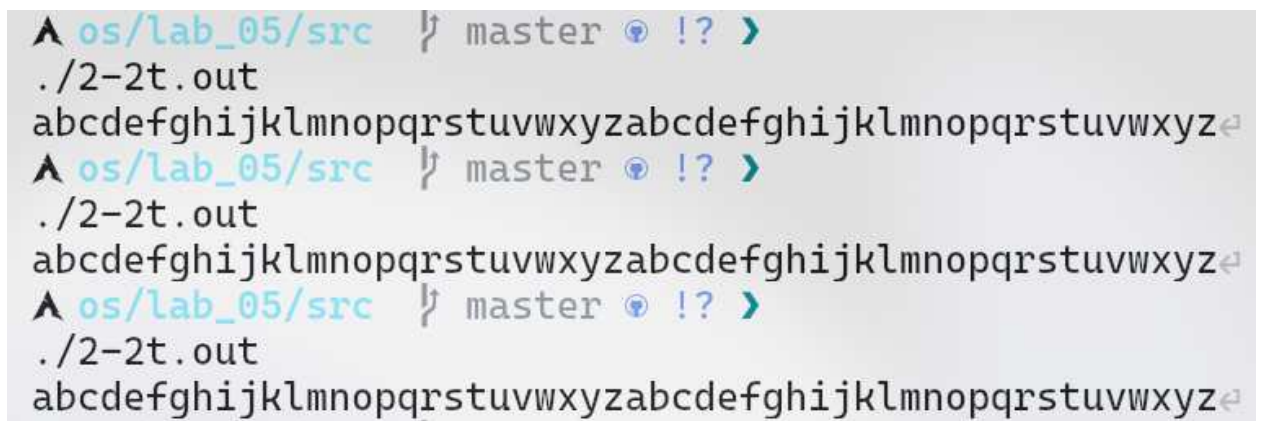
```
1 #include <fcntl.h>
2 #include <pthread.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6
7 void *thread_func(void *arg)
8 {
9     char c;
10    int fl = 1;
11    int fd = *((int *)arg);
12
13    while (fl == 1)
14    {
15        if ((fl = read(fd, &c, 1)))
16            write(1, &c, 1);
17    }
18    return NULL;
19 }
20
21 int main()
22 {
23     pthread_t t;
24     char c;
25     int fl = 1;
26
27     int fd1 = open("alphabet.txt", O_RDONLY);
28     int fd2 = open("alphabet.txt", O_RDONLY);
29
30     if (pthread_create(&t, NULL, thread_func, &fd1) != 0)
31     {
32         perror("pthread_create");
33         exit(1);
34     }
35
36     while (fl == 1)
```

Листинг 5 (продолжение)

```
37     {
38         if ((fl = read(fd2, &c, 1)) == 1)
39             write(1, &c, 1);
40     }
41     pthread_join(t, NULL);
42     return 0;
43 }
```

3.2.2 Результат работы

Результат работы второй программы с одним дополнительным потоком представлен на рисунке 4.



```
A os/lab_05/src  master  !? >
./2-2t.out
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz
A os/lab_05/src  master  !? >
./2-2t.out
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz
A os/lab_05/src  master  !? >
./2-2t.out
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz
```

Рисунок 4 – Результат работы второй программы с одним дополнительным потоком

3.2.3 Анализ полученного результата

В данной версии программы главный поток первоначально выполняет два независимых системных вызова `open()`, получая два уникальных файловых дескриптора. В ядре операционной системы, как и в однопоточном варианте, создаются две независимые структуры `struct file`, каждая из которых ссылается на один и тот же `inode` и инициализируется собственным смещением `f_pos = 0`.

Затем главный поток с помощью библиотечной функции `pthread_create()` порождает один дополнительный поток выполнения, передавая ему указатель на первый файловый дескриптор. Главный поток при этом не блокируется и продолжает свое исполнение, используя для чтения второй файловый

дескриптор. Блокировка главного потока функцией `pthread_join()` происходит только после завершения его собственного цикла чтения.

Оба потока параллельно и независимо друг от друга выполняют системные вызовы `read()`. Поскольку каждый поток оперирует собственной структурой `struct file`, состояние гонки за позицию чтения внутри файла исключено: смещения независимы, и каждый из потоков гарантированно прочитает весь файл от начала до конца.

Тем не менее, вывод прочитанных символов осуществляется через системный вызов `write()` в стандартный поток вывода, который выступает общим разделяемым ресурсом для всего процесса. Поскольку создание нового потока сопряжено с системными накладными расходами ядра, главный поток может успеть прочитать и вывести некоторую часть данных, в том числе и весь файл до фактического выделения процессорного времени дополнительному потоку.

После создания дополнительного потока чтение и вывод символов осуществляются обоими потоками в зависящей от планировщика последовательности.

Когда главный поток завершит чтение по своему дескриптору, он выйдет из цикла и заблокируется в вызове `pthread_join`, ожидая завершения дополнительного потока. Дополнительный поток продолжит работу и выведет оставшуюся часть своих символов.

Таким образом, каждый символ файла будет напечатан дважды, однако точный порядок их вывода не определён и полностью зависит от того, как планировщик распределяет процессорное время между потоками.

3.3 Два дополнительных потока

3.3.1 Исходный код

Исходный код второй программы с двумя дополнительными потоками представлен в листинге 6.

Листинг 6 – Исходный код второй программы с двумя дополнительными потоками

```
1 #include <fcntl.h>
2 #include <pthread.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6
7 void *thread_func(void *arg)
8 {
9     char c;
10    int fl = 1;
11    int fd = *((int *)arg);
12
13    while (fl == 1)
14    {
15        if ((fl = read(fd, &c, 1)))
16            write(1, &c, 1);
17    }
18    return NULL;
19 }
20
21 int main()
22 {
23    pthread_t t1, t2;
24
25    int fd1 = open("alphabet.txt", O_RDONLY);
26    int fd2 = open("alphabet.txt", O_RDONLY);
27
28    if (pthread_create(&t1, NULL, thread_func, &fd1) != 0)
29    {
30        perror("pthread_create");
31        exit(1);
32    }
33    if (pthread_create(&t2, NULL, thread_func, &fd2) != 0)
34    {
35        perror("pthread_create");
36        exit(1);
37    }
38
39    pthread_join(t1, NULL);
40    pthread_join(t2, NULL);
41
42    return 0;
43 }
```

3.3.2 Результат работы

Результат работы второй программы с двумя дополнительными потоками представлен на рисунке 4.



```
os/lab_05/src master !? >
./2-3t.out
abcdefghijklmnopqrstuvwxyz
os/lab_05/src master !? >
./2-3t.out
aabbccddeeffgghhiijjkkllmmnnooppqrrssttuuvvwwxxyyzz
os/lab_05/src master !? >
./2-3t.out
abcdefagbhcidjekflgmhniojpkqlrmsntoupvqwrxsytzuvwxyz
os/lab_05/src master !? >
./2-3t.out
abcdefghijklmnopqrstuvwxyz
```

Рисунок 5 – Результат работы второй программы с двумя дополнительными потоками

3.3.3 Анализ полученного результата

В этой реализации ответственность за открытие файлов возложена исключительно на главный поток: до создания дополнительных потоков он дважды выполняет системный вызов `open()`, получая два файловых дескриптора. На каждый вызов в системной таблице открытых файлов ядра создается отдельная структура `struct file` с нулевым начальным смещением.

После этого главный поток с помощью вызовов `pthread_create()` создает два дополнительных рабочих потока, передавая каждому из них по одному уникальному дескриптору. Сразу после создания потоков главный поток переходит в заблокированное состояние посредством вызовов `pthread_join()`, ожидая завершения работы своих потомков.

С точки зрения структур ядра логика работы с файлом остается идентичной предыдущему варианту. Два рабочих потока, используя выделенные им дескрипторы, независимо и параллельно выполняют системные вызовы `read()`, считывая данные от начала до конца файла. Конфликтов за указатель текущей позиции не происходит, так как каждый дескриптор

привязан к своей структуре struct file.

Вывод прочитанных символов осуществляется конкурентно через вызовы write() в общий стандартный вывод. Поскольку порядок исполнения потоков, их старт и выделяемые им кванты процессорного времени полностью контролируются планировщиком ОС, точная очередность вывода символов не может быть предсказана. Вследствие параллельной работы потоков и отсутствия механизмов синхронизации при обращении к stdout, итоговый результат детерминирован лишь по составу данных (каждый символ исходного файла будет напечатан ровно дважды), но на экране формируется непредсказуемо перемешанная последовательность из двух алфавитов.

3.4 Схема связи структур

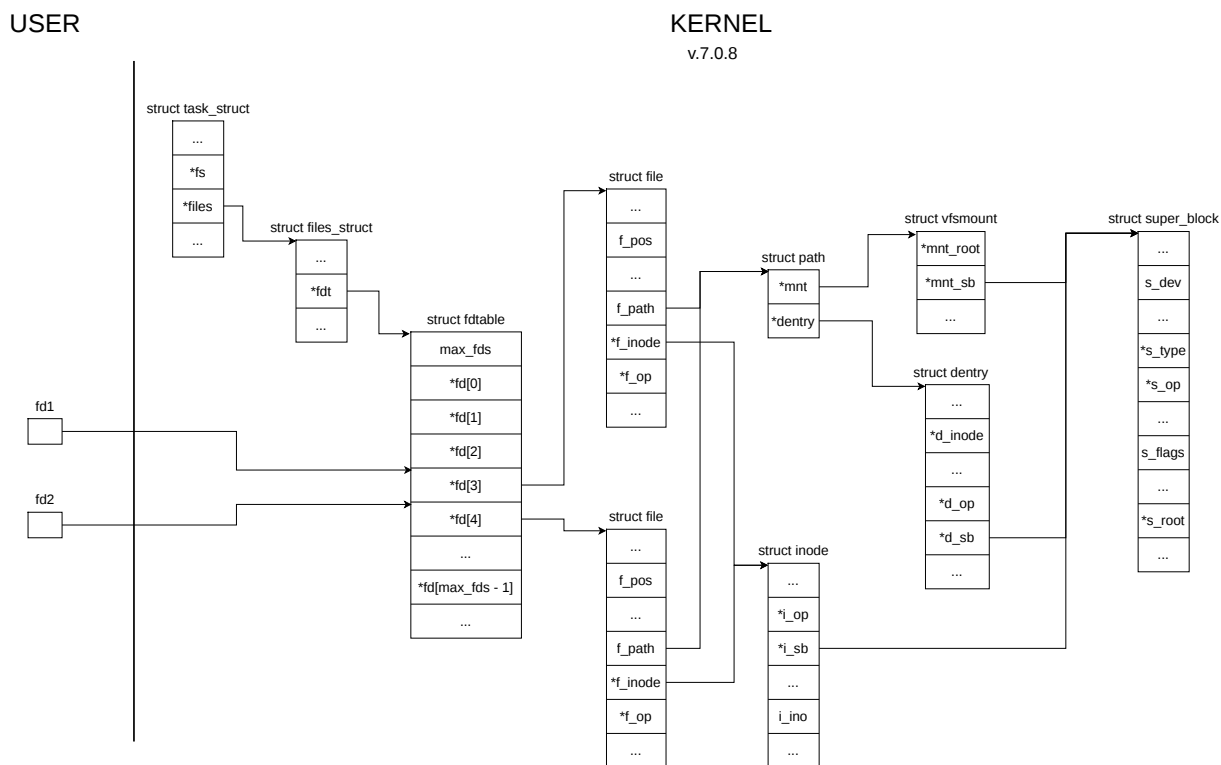


Рисунок 6 – Схема связи структур в первой программе

4 Третья программа

4.1 Небуферизованный ввод-вывод

4.1.1 Исходный код

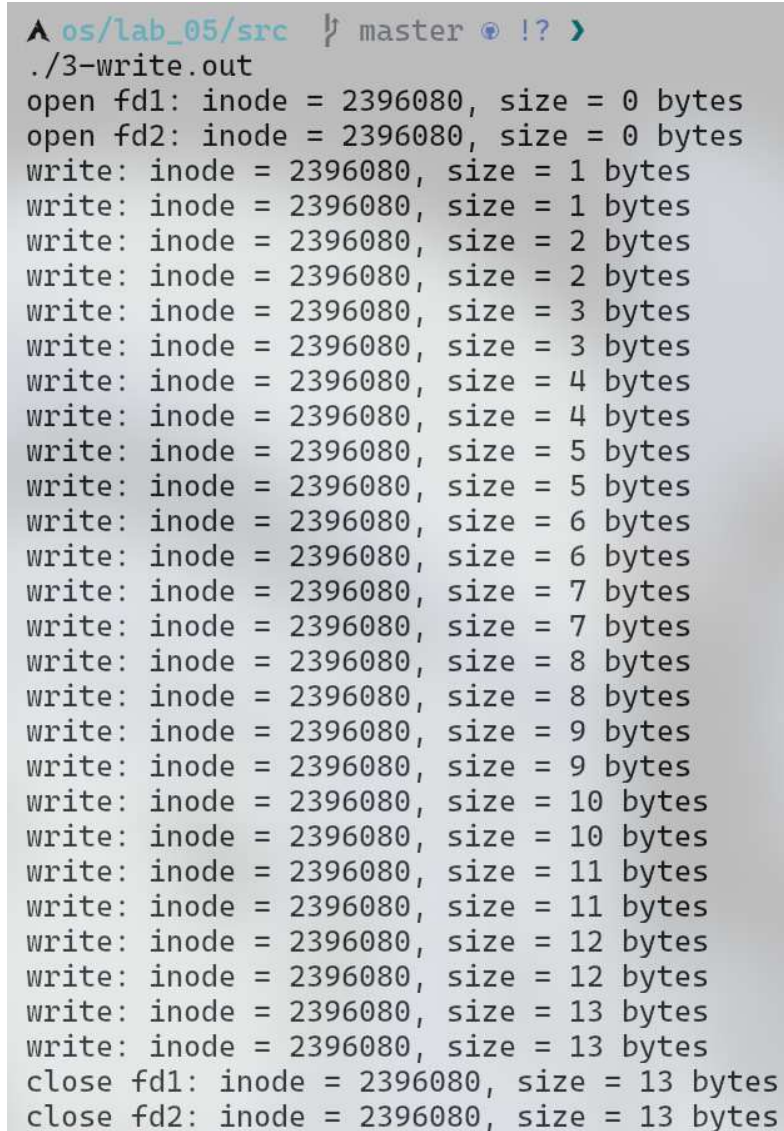
Исходный код третьей программы с небуферизованным вводом-выводом представлен в листинге 7.

Листинг 7 – Исходный код третьей программы с небуферизованным вводом-выводом

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <sys/stat.h>
4 #include <stdio.h>
5
6 void print_stat(char *msg)
7 {
8     struct stat statbuf;
9     stat("q.txt", &statbuf);
10    printf("%s: inode = %lu, size = %ld bytes\n", msg,
11           statbuf.st_ino, statbuf.st_size);
12 }
13
14 int main()
15 {
16     int fd1 = open("q.txt", O_RDWR);
17     print_stat("open fd1");
18     int fd2 = open("q.txt", O_RDWR);
19     print_stat("open fd2");
20     for (char c = 'a'; c <= 'z'; c++)
21     {
22         if (c % 2)
23             write(fd1, &c, 1);
24         else
25             write(fd2, &c, 1);
26         print_stat("write");
27     }
28     close(fd1);
29     print_stat("close fd1");
30     close(fd2);
31     print_stat("close fd2");
32     return 0;
33 }
```

4.1.2 Результат работы

Результат работы третьей программы с небуферизованным вводом-выводом представлен на рисунке 7.



```
os/lab_05/src master !? >
./3-write.out
open fd1: inode = 2396080, size = 0 bytes
open fd2: inode = 2396080, size = 0 bytes
write: inode = 2396080, size = 1 bytes
write: inode = 2396080, size = 1 bytes
write: inode = 2396080, size = 2 bytes
write: inode = 2396080, size = 2 bytes
write: inode = 2396080, size = 3 bytes
write: inode = 2396080, size = 3 bytes
write: inode = 2396080, size = 4 bytes
write: inode = 2396080, size = 4 bytes
write: inode = 2396080, size = 5 bytes
write: inode = 2396080, size = 5 bytes
write: inode = 2396080, size = 6 bytes
write: inode = 2396080, size = 6 bytes
write: inode = 2396080, size = 7 bytes
write: inode = 2396080, size = 7 bytes
write: inode = 2396080, size = 8 bytes
write: inode = 2396080, size = 8 bytes
write: inode = 2396080, size = 9 bytes
write: inode = 2396080, size = 9 bytes
write: inode = 2396080, size = 10 bytes
write: inode = 2396080, size = 10 bytes
write: inode = 2396080, size = 11 bytes
write: inode = 2396080, size = 11 bytes
write: inode = 2396080, size = 12 bytes
write: inode = 2396080, size = 12 bytes
write: inode = 2396080, size = 13 bytes
write: inode = 2396080, size = 13 bytes
close fd1: inode = 2396080, size = 13 bytes
close fd2: inode = 2396080, size = 13 bytes
```

Рисунок 7 – Результат работы второй программы с небуферизованным вводом-выводом

4.1.3 Анализ полученного результата

В данном варианте программы один и тот же целевой файл открывается дважды системным вызовом `open()` в режиме чтения и записи (`O_RDWR`). В результате этого в таблице открытых файлов процесса выделяются два уникальных целочисленных дескриптора (`fd1` и `fd2`). На уровне ядра операционной системы создаются две независимые структуры

struct file, обе из которых ссылаются на один и тот же inode открываемого файла. Важнейшим аспектом является то, что каждая из этих структур обладает собственным, независимым указателем смещения f_pos, который инициализируется нулем.

В цикле программа поочередно выполняет системные вызовы write() для каждого из дескрипторов. При первой итерации вызов write() через дескриптор fd1 записывает первый символ по нулевому смещению. После выполнения операции f_pos первой структуры struct file инкрементируется на 1 байт. Текущий размер файла на диске становится равным 1 байту.

Однако при следующей итерации системный вызов write() выполняется уже для дескриптора fd2. Поскольку его ассоциированная структура struct file независима, ее указатель f_pos всё еще равен нулю. Следовательно, запись нового символа происходит с нулевой позиции, физически перезаписывая ранее сохраненный символ. После этой операции f_pos второй структуры также увеличивается на 1, а размер файла остается равным 1 байту.

Такая конкурентная поочередная запись продолжается до конца цикла. Из-за наличия двух независимых, несинхронизированных указателей файлового смещения, данные, передаваемые ядру через fd2, систематически перезаписывают информацию, только что записанную через fd1. В результате происходит потеря данных: итоговый файл имеет размер всего 13 байт и содержит только те символы, которые были выведены через второй файловый дескриптор, то есть каждую букву алфавита через одну позицию, начиная с «b».

Во избежание потерь данных при записи необходимо использовать флаг O_APPEND при использовании функции open.

4.2 Буферизованный ввод-вывод

4.2.1 Исходный код

Исходный код второй программы с буферизованным вводом-выводом представлен в листинге 8.

Листинг 8 – Исходный код второй программы с буферизованным ВВОДОМ-ВЫВОДОМ

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <sys/stat.h>
4
5 void print_stat(char *msg)
6 {
7     struct stat statbuf;
8     stat("q.txt", &statbuf);
9     printf("%s: inode = %ld, size = %ld bytes\n", msg,
10         statbuf.st_ino, statbuf.st_size);
11 }
12 int main()
13 {
14     FILE *fs1 = fopen("q.txt", "w");
15     print_stat("fopen fs1");
16     FILE *fs2 = fopen("q.txt", "w");
17     print_stat("fopen fs2");
18     for (char c = 'a'; c <= 'z'; c++)
19     {
20         if (c % 2)
21             fprintf(fs1, "%c", c);
22         else
23             fprintf(fs2, "%c", c);
24         print_stat("fprintf");
25     }
26     fclose(fs1);
27     print_stat("fclose fs1");
28     fclose(fs2);
29     print_stat("fclose fs2");
30     return 0;
31 }
```

4.2.2 Результат работы

Результат работы второй программы с буферизованным вводом-выводом представлен на рисунке 8.

4.2.3 Анализ полученного результата

В этой версии программы целевой файл дважды открывается библиотечной функцией `fopen()` в режиме «только для записи». Аналогично

в оперативной памяти процесса формируются два независимых массива данных по 13 байт каждый, при этом реальный файл на диске остается пустым.

Сброс накопленной информации из адресного пространства пользователя на физический носитель происходит только при закрытии потоков функцией `fclose()`. Когда выполняется вызов `fclose(fs1)`, библиотека инициирует системный вызов `write()`, который единым блоком записывает 13 байт из первого буфера в файл. Запись осуществляется с позиции `f_pos = 0` первой структуры `struct file`, после чего её смещение сдвигается на 13 байт.

Следом выполняется вызов `fclose(fs2)`. Он также инициирует системный вызов `write()` для сброса 13 байт из второго буфера. Однако, поскольку вторая структура `struct file` абсолютно независима и её смещение `f_pos` всё ещё равно нулю, ядро записывает эти 13 байт с самого начала файла. Это приводит к полной перезаписи всего блока данных, только что сохраненного вызовом `fclose(fs1)`. В итоге на выходе формируется файл размером ровно 13 байт, содержащий исключительно последовательность символов из буфера, ассоциированного с `fs2`.

Во избежание потерь данных при записи необходимо использовать флаг «а» при использовании функции `fopen`.

4.3 Схема связи структур

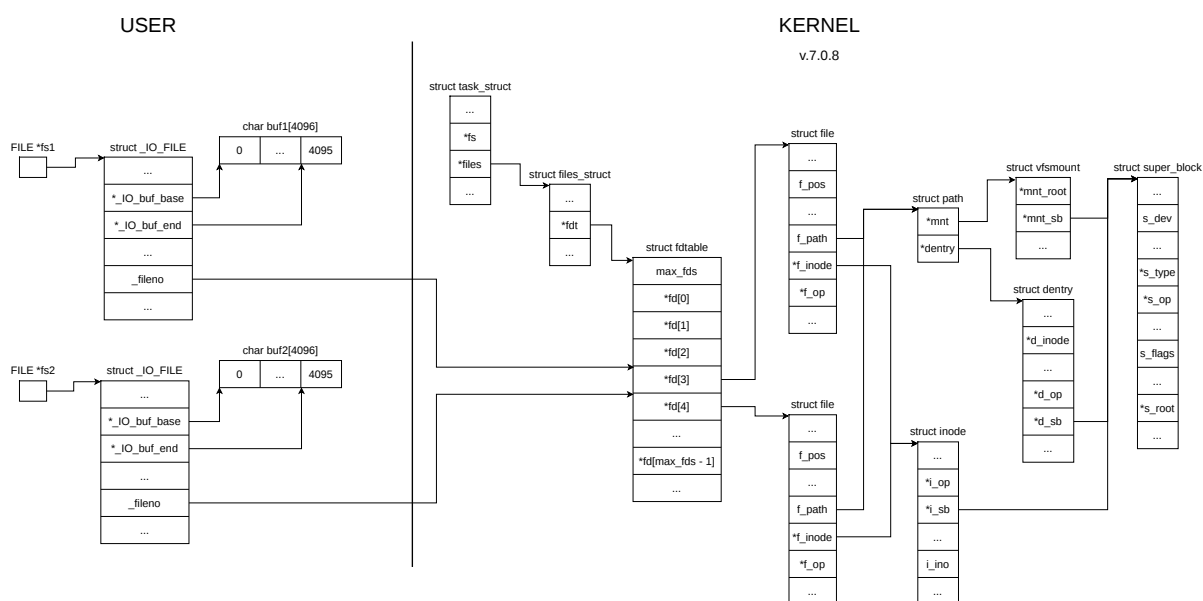


Рисунок 9 – Схема связи структур в первой программе